# HayatApp - Frontend & Integration Architecture Guide

Complete React Single Page Application (SPA) with Socket.io & WebRTC Integration

React + Vite | Render Backend | Vercel Ready

> **🔗 Complete Connection Blueprint**
>
> This document contains the production-ready code for **HayatApp Frontend**. It connects your Vercel-hosted React app with your Render backend (**https://hayaty-backend.onrender.com**) supporting Chatting, WebRTC Signaling (Voice/Video), and Vercel routing configuration.

## 1. Project Setup & Package Installation

Run the following commands in your terminal to initialize your React Vite project and install required dependencies:

**Terminal Command**

```
# Create React project with Vite
npm create vite@latest hayatapp-frontend -- --template react
cd hayatapp-frontend

# Install core dependencies for Real-time Chat & WebRTC
npm install socket.io-client axios lucide-react

# Run dev server
npm run dev
```

## 2. Recommended File Structure

| File Name | Purpose / Role |
|---|---|
| **vercel.json** | Prevents 404 errors on Vercel during SPA route refreshes. |
| **src/config.js** | Holds Backend Render URL and Socket configurations. |
| **src/App.jsx** | Main WhatsApp-style Chat UI with Voice/Video Call Modal. |
| **src/index.css** | Modern WhatsApp dark theme CSS styling. |

## 3. Vercel Configuration (vercel.json)

Place this file in the root folder of your project before pushing to GitHub and deploying to Vercel:

**vercel.json**

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

## 4. Central Configuration (src/config.js)

This file connects directly to your live Render backend URL:

**src/config.js**

```
import { io } from "socket.io-client";

export const BACKEND_URL = "https://hayaty-backend.onrender.com";

// Initialize Socket.io with automatic transport selection
export const socket = io(BACKEND_URL, {
  transports: ["websocket", "polling"],
  autoConnect: false,
});
```

## 5. Main App Component (src/App.jsx)

Includes complete WhatsApp UI layout, WebSocket live chat, and WebRTC peer-to-peer call handler:

**src/App.jsx**

```
import React, { useState, useEffect, useRef } from "react";
import { socket, BACKEND_URL } from "./config";

export default function App() {
  const [userId, setUserId] = useState("User_" + Math.floor(Math.random() * 1000));
  const [connected, setConnected] = useState(false);
  const [messages, setMessages] = useState([]);
  const [inputText, setInputText] = useState("");
  const [activeCall, setActiveCall] = useState(false);
  const [callType, setCallType] = useState(null); // 'voice' or 'video'

  const localVideoRef = useRef(null);
  const remoteVideoRef = useRef(null);
  const peerConnectionRef = useRef(null);

  useEffect(() => {
    socket.auth = { userId };
    socket.connect();

    socket.on("connect", () => setConnected(true));
    socket.on("disconnect", () => setConnected(false));

    socket.on("receive_message", (data) => {
      setMessages((prev) => [...prev, data]);
    });

    socket.on("webrtc_signal", async (data) => {
      if (data.type === "offer") {
        await handleReceiveOffer(data.offer);
      } else if (data.type === "answer") {
        await peerConnectionRef.current?.setRemoteDescription(new RTCSessionDescription(data.answer));
      } else if (data.candidate) {
```

```
        await peerConnectionRef.current?.addIceCandidate(new RTCIceCandidate(data.candidate));
      }
    });

    return () => {
      socket.disconnect();
    };
  }, []);

  const handleSendMessage = (e) => {
    e.preventDefault();
    if (!inputText.trim()) return;

    const msgData = { sender: userId, text: inputText, time: new Date().toLocaleTimeString([], { hour:
'2-digit', minute: '2-digit' }) };
    socket.emit("send_message", msgData);
    setMessages((prev) => [...prev, msgData]);
    setInputText("");
  };

  const startCall = async (type) => {
    setCallType(type);
    setActiveCall(true);

    const pc = new RTCPeerConnection({
      iceServers: [{ urls: "stun:stun.l.google.com:19302" }]
    });
    peerConnectionRef.current = pc;

    const stream = await navigator.mediaDevices.getUserMedia({
      video: type === "video",
      audio: true
    });
    if (localVideoRef.current) localVideoRef.current.srcObject = stream;

    stream.getTracks().forEach((track) => pc.addTrack(track, stream));

    pc.onicecandidate = (event) => {
      if (event.candidate) {
        socket.emit("webrtc_signal", { candidate: event.candidate });
      }
    };

    pc.ontrack = (event) => {
      if (remoteVideoRef.current) remoteVideoRef.current.srcObject = event.streams[0];
    };

    const offer = await pc.createOffer();
    await pc.setLocalDescription(offer);
    socket.emit("webrtc_signal", { type: "offer", offer });
  };

  const handleReceiveOffer = async (offer) => {
    setActiveCall(true);
    const pc = new RTCPeerConnection({
      iceServers: [{ urls: "stun:stun.l.google.com:19302" }]
    });
    peerConnectionRef.current = pc;

    const stream = await navigator.mediaDevices.getUserMedia({ video: true, audio: true });
    if (localVideoRef.current) localVideoRef.current.srcObject = stream;

    stream.getTracks().forEach((track) => pc.addTrack(track, stream));

    await pc.setRemoteDescription(new RTCSessionDescription(offer));
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
```

```
    socket.emit("webrtc_signal", { type: "answer", answer });
  };

  const endCall = () => {
    peerConnectionRef.current?.close();
    setActiveCall(false);
  };

  return (
    <div style={{ display: 'flex', height: '100vh', backgroundColor: '#0b141a', color: '#e9edef' }}>
      <div style={{ width: '300px', borderRight: '1px solid #222d34', padding: '15px' }}>
        <h3 style={{ color: '#00a884' }}>HayatApp</h3>
        <p>Status: <span style={{ color: connected ? '#00a884' : '#ef4444' }}>{connected ? "Online" :
"Connecting..."}</span></p>
        <p style={{ fontSize: '12px', color: '#8696a0' }}>Your ID: {userId}</p>
      </div>

      <div style={{ flex: 1, display: 'flex', flexDirection: 'column' }}>
        <div style={{ padding: '15px', backgroundColor: '#202c33', display: 'flex', justifyContent:
'space-between' }}>
          <span style={{ fontWeight: 'bold' }}>Global Chat Room</span>
          <div>
            <button onClick={() => startCall('voice')} style={{ marginRight: '10px', background:
'#00a884', border: 'none', padding: '6px 12px', color: '#fff', borderRadius: '4px' }}>📞 Voice Call</
button>
            <button onClick={() => startCall('video')} style={{ background: '#00a884', border: 'none',
padding: '6px 12px', color: '#fff', borderRadius: '4px' }}>📹 Video Call</button>
          </div>
        </div>

        <div style={{ flex: 1, padding: '20px', overflowY: 'auto', background: '#0b141a' }}>
          {messages.map((m, idx) => (
            <div key={idx} style={{ textAlign: m.sender === userId ? 'right' : 'left', margin: '8px 0' }}
>
              <span style={{ display: 'inline-block', background: m.sender === userId ? '#005c4b' :
'#202c33', padding: '8px 12px', borderRadius: '8px', maxWidth: '60%' }}>
                <div style={{ fontSize: '10px', color: '#8696a0' }}>{m.sender}</div>
                {m.text}
                <div style={{ fontSize: '9px', color: '#8696a0', textAlign: 'right' }}>{m.time}</div>
              </span>
            </div>
          ))}
        </div>

        <form onSubmit={handleSendMessage} style={{ padding: '10px', backgroundColor: '#202c33', display:
'flex' }}>
          <input type="text" value={inputText} onChange={(e) => setInputText(e.target.value)}
placeholder="Type a message..." style={{ flex: 1, padding: '10px', background: '#2a3942', border: 'none',
color: '#fff', borderRadius: '8px', marginRight: '10px' }} />
          <button type="submit" style={{ background: '#00a884', border: 'none', padding: '10px 20px',
color: '#fff', borderRadius: '8px' }}>Send</button>
        </form>
      </div>

      {activeCall && (
        <div style={{ position: 'fixed', top: 0, left: 0, right: 0, bottom: 0, background:
'rgba(0,0,0,0.9)', display: 'flex', flexDirection: 'column', alignItems: 'center', justifyContent:
'center', zIndex: 1000 }}>
          <h2 style={{ color: '#fff' }}>HayatApp {callType === 'video' ? 'Video' : 'Voice'} Call</h2>
          <div style={{ display: 'flex', gap: '20px', margin: '20px 0' }}>
            <video ref={localVideoRef} autoPlay muted style={{ width: '250px', borderRadius: '10px',
border: '2px solid #00a884' }} />
            <video ref={remoteVideoRef} autoPlay style={{ width: '250px', borderRadius: '10px', border:
'2px solid #38bdf8' }} />
          </div>
          <button onClick={endCall} style={{ background: '#ef4444', border: 'none', padding: '12px 24px',
```

```
color: '#fff', borderRadius: '25px', fontSize: '16px', cursor: 'pointer' }}>End Call</button>
        </div>
      )}
    </div>
  );
}
```

## 6. GitHub Push & Vercel Deployment Steps

1. **Step 1:** Create a new empty repository on GitHub named **hayatapp-frontend**.
2. **Step 2:** Run terminal commands to push your project:

```
git init
git add .
git commit -m "Initial commit of HayatApp frontend"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/hayatapp-frontend.git
git push -u origin main
```

3. **Step 3:** Go to **vercel.com**, click **"Add New Project"**, and import **hayatapp-frontend**.
4. **Step 4:** Click **Deploy**. Vercel will automatically build and publish your app with an SSL certificate (**https://hayatapp-frontend.vercel.app**).

## 7. How to Test HayatApp End-to-End

- Open your Vercel link in two different browser windows (or one mobile phone and one laptop).
- Check the online indicator status—it will automatically turn **Online** once connected to **https://hayaty-backend.onrender.com**.
- Type messages to test real-time chat sync.
- Click **Voice Call** or **Video Call** to test peer-to-peer WebRTC audio/video stream.